



The sciences have identified restrictions on

when (src) {  **src.balance += 10; }**

when (dst) {  **dst.balance += 10; }**



when(dst) {  **print(dst.balance) }**

when(src) {  **print(src.balance) }**

src=0, dst=0



src=0, dst=10



src=10, dst=0



src=10, dst=10



Order

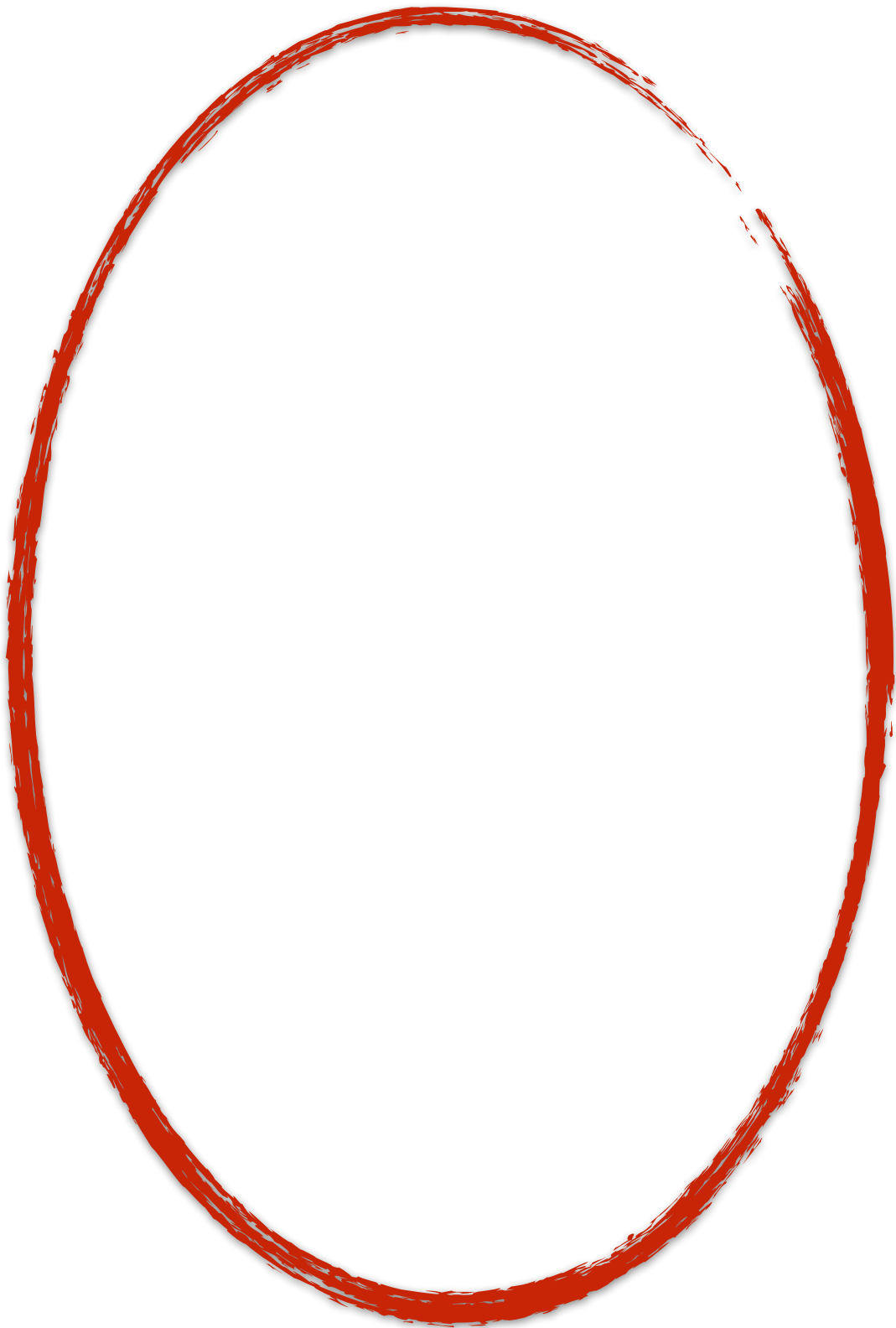
Permitted





updatepath(src then dst)

Diagnostic path (dst then src)



The semantics have incidental restrictions on executions

Update path (src then dst)

when(src) { A src.balance += 10; }

when(dst) { B dst.balance += 10; }

Diagnostic path (dst then src)

when(dst) { C print(dst.balance) }

when(src) { D print(src.balance) }

	src=0, dst=0	src=0, dst=10	src=10, dst=0	src=10, dst=10
Order	CBDA	BCDA	ACBD	ABCD
	CDAB	BDAC	ACDB	ABDC
	CDBA	BDCA	ADCB	ADBC
	DACB	DABC	CABD	BACD
	DCBA	DCBA	CBAD	BCAD
Permitted	✓	⚠	✓	✓

There are restrictions we want to keep

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    }  
  }  
}
```

A nested behaviour is spawned synchronously,
but can run independently of it's parent